

WEEK-7 ASSIGNMENT SCREENSHOTS

```
▶ ✓ Just now (<1s) 1
DATA_PATH = "dbacademy.default.raw_data"
DB_NAME = "delta_scd_assignment"

▶ ✓ Just now (<1s) 2
MASTER_FILE = f"{DATA_PATH}/customer_master.csv"
INCREMENTAL_FILE = f"{DATA_PATH}/customer_incremental.csv"

▶ ✓ Just now (2s) 3
spark.sql(f"CREATE DATABASE IF NOT EXISTS {DB_NAME}")
spark.sql(f"USE {DB_NAME}")
> See performance \(2\) ✓ Checked for improvements
DataFrame[]
```

```
▶ ✓ Just now (<1s) 4
print(f"Using database: {DB_NAME}")
print(f"Master file: {MASTER_FILE}")
print(f"Incremental file: {INCREMENTAL_FILE}")

Using database: delta_scd_assignment
Master file: dbacademy.default.raw_data/customer_master.csv
Incremental file: dbacademy.default.raw_data/customer_incremental.csv

▶ ✓ Just now (1s) 5
from pyspark.sql.types import StructType, StructField, StringType,
IntegerType
from pyspark.sql import functions as F
```

✓ 4 minutes ago (<1s)

6

```
schema = StructType([
    StructField("customer_id", IntegerType(), True),
    StructField("first_name", StringType(), True),
    StructField("last_name", StringType(), True),
    StructField("email", StringType(), True),
    StructField("city", StringType(), True),
    StructField("phone", StringType(), True),
    StructField("updated_at", StringType(), True),
])
```

✓ 4 minutes ago (16s)

7

```
raw_df = (
    spark.read
    .option("header", True)
    .schema(schema)
    .csv("/Volumes/dbacademy/default/raw_data/customer_master.csv")
)
```

```
print(f"Raw row count: {raw_df.count()}")
display(raw_df)
```

> [See performance \(2\)](#)

✓ Checked for improvements

> raw_df: pyspark.sql.connect.dataframe.DataFrame = [customer_id: integer, first_name: string ... 5 more fields]

Raw row count: 16

	customer_id	first_name	last_name	email	city	phone	updated_at
2	1002	Jane	Smith	jane.smith@email.com	Los Angeles	555-0102	2024-01-16
3	1003	Robert	Brown	null	Chicago	555-0103	2024-01-17
4	1004	Emily	Davis	emily.davis@email.com	Houston	555-0104	2024-01-18
5	1005	Michael	Wilson	michael.wilson@email.com	Phoenix	555-0105	2024-01-19
6	1001	John	Doe	john.doe@email.com	New York	555-0101	2024-01-15
7	1006	Sarah	Taylor	sarah.taylor@email.com	null	555-0106	2024-01-20

✓ 5 minutes ago (9s)

8

```
(
    raw_df.write
    .format("delta")
    .mode("overwrite")
    .saveAsTable("customers_bronze")
)

print("customers_bronze row count:", spark.table("customers_bronze").count())
```

> [See performance \(2\)](#)

✓ Checked for improvements

customers_bronze row count: 16

```
08:40 AM (3s) 9

bronze_df = spark.table("customers_bronze")

print("Null counts per column (before cleaning):")
bronze_df.select([
    F.count(F.when(F.col(c).isNull(), c)).alias(c) for c in bronze_df.columns
]).show()
dup_count_before = bronze_df.count() - bronze_df.dropDuplicates().count()
print(f"Exact duplicate rows found: {dup_count_before}")

> See performance \(3\) ✓ Checked for improvements

>  bronze_df: pyspark.sql.connect.dataframe.DataFrame = [customer_id: integer, first_name: string ... 5 more fields]

Null counts per column (before cleaning):
+-----+-----+-----+-----+-----+-----+
|customer_id|first_name|last_name|email|city|phone|updated_at|
+-----+-----+-----+-----+-----+-----+
|          0|          0|          0|      2|    1|      0|          0|
+-----+-----+-----+-----+-----+-----+

Exact duplicate rows found: 1
```

```
08:40 AM (3s) 10

cleaned_df = (
    bronze_df
    .dropDuplicates()
    .dropna(subset=["customer_id"])
    .fillna({"email": "unknown@example.com", "city": "Unknown"})
    .withColumn("updated_at", F.to_date("updated_at"))
    .dropDuplicates(["customer_id"])
)

print(f"Row count after cleaning: {cleaned_df.count()}")
display(cleaned_df)

> See performance \(2\) ✓ Checked for improvements

>  cleaned_df: pyspark.sql.connect.dataframe.DataFrame = [customer_id: integer, first_name: string ... 5 more fields]

Row count after cleaning: 15

Table +    
+-----+-----+-----+-----+-----+-----+-----+
|customer_id|first_name|last_name|email|city|phone|updated_at|
+-----+-----+-----+-----+-----+-----+
|1|1001|John|Doe|john.doe@email.com|New York|555-0101|2024-01-15|
+-----+-----+-----+-----+-----+-----+-----+
```

```
08:40 AM (5s) 11

(
    cleaned_df.write
    .format("delta")
    .mode("overwrite")
    .saveAsTable("customers_silver")
)

> See performance \(1\) ✓ Checked for improvements

08:40 AM (1s) 12

print("customers_silver row count:", spark.table("customers_silver").count())

> See performance \(1\) ✓ Checked for improvements

customers_silver row count: 15

08:40 AM (2s) 13
```

08:40 AM (2s)

13

```
incremental_df = (  
    spark.read  
    .option("header", True)  
    .schema(schema)  
    .csv("/Volumes/dbacademy/default/raw_data/customer_incremental.csv")  
    .withColumn("updated_at", F.to_date("updated_at"))  
)  
  
print(f"Incremental row count: {incremental_df.count()}")  
display(incremental_df)
```

> [See performance \(2\)](#)

✓ Checked for improvements

> incremental_df: pyspark.sql.connect.dataframe.DataFrame = [customer_id: integer, first_name: string ... 5 more fields]

Incremental row count: 8

Table +

🔍 🔍 📄 🗑️

	customer_id	first_name	last_name	email	city	phone	updated_at
1	1002	Jane	Smith	jane.smith@newmail.com	San Francisco	555-0102	2024-02-10
2	1004	Emily	Davis	emily.davis@newmail.com	Houston	555-0119	2024-02-16
3	1005	Michael	Wilson	michael.wilson@email.co...	Denver	555-0105	2024-02-11
4	1009	James	Jackson	james.jackson@email.com	Seattle	555-0109	2024-02-12
5	1011	William	Harris	william.harris@email.com	Sacramento	555-0111	2024-02-17

08:40 AM (2s)

14

```
existing_ids = set(r.customer_id for r in spark.table("customers_silver").select("customer_id").collect())  
incoming_ids = set(r.customer_id for r in incremental_df.select("customer_id").collect())  
  
updates = incoming_ids & existing_ids  
inserts = incoming_ids - existing_ids  
  
print(f"Incoming records that UPDATE existing customers: {sorted(updates)}")  
print(f"Incoming records that INSERT new customers: {sorted(inserts)}")
```

> [See performance \(2\)](#)

✓ Checked for improvements

Incoming records that UPDATE existing customers: [1002, 1004, 1005, 1009, 1011]

Incoming records that INSERT new customers: [1016, 1017, 1018]

08:40 AM (<1s)

15

```
from delta.tables import DeltaTable
```

▶

08:40 AM (1s)

16

```
if not spark.catalog.tableExists(f"{DB_NAME}.customers_scd1"):
    spark.table("customers_silver").write.format("delta").saveAsTable("customers_scd1")

scd1_table = DeltaTable.forName(spark, "customers_scd1")
```

>

[See performance \(1\)](#)

✓ Checked for improvements

▶

08:40 AM (11s)

17

```
(
    scd1_table.alias("target")
    .merge(
        incremental_df.alias("source"),
        "target.customer_id = source.customer_id"
    )
    .whenMatchedUpdateAll() # overwrite existing customer with latest values
    .whenNotMatchedInsertAll() # insert brand-new customers
    .execute()
)
```

>

[See performance \(1\)](#)

✓ Checked for improvements

DataFrame[num_affected_rows: bigint, num_updated_rows: bigint, num_deleted_rows: bigint, num_inserted_rows: bigint]

▶

08:40 AM (3s)

18

```
print("customers_scd1 row count after merge:", spark.table("customers_scd1").count())
display(spark.table("customers_scd1").orderBy("customer_id"))
```

>

[See performance \(2\)](#)

✓ Checked for improvements

customers_scd1 row count after merge: 18

Table								
	customer_id	first_name	last_name	email	city	phone	updated_at	
1	1001	John	Doe	john.doe@email.com	New York	555-0101	2024-01-15	
2	1002	Jane	Smith	jane.smith@newmail.com	San Francisco	555-0102	2024-02-10	
3	1003	Robert	Brown	unknown@example.com	Chicago	555-0103	2024-01-17	
4	1004	Emily	Davis	emily.davis@newmail.com	Houston	555-0119	2024-02-16	
5	1005	Michael	Wilson	michael.wilson@email.com	Denver	555-0105	2024-02-11	
6	1006	Sarah	Taylor	sarah.taylor@email.com	Unknown	555-0106	2024-01-20	
7	1007	David	Anderson	david.anderson@email.com	Philadelphia	555-0107	2024-01-21	
8	1008	Laura	Thomas	unknown@example.com	San Antonio	555-0108	2024-01-22	
9	1009	James	Jackson	james.jackson@email.com	Seattle	555-0109	2024-02-12	

▶

08:40 AM (1s)

19

```
if not spark.catalog.tableExists(f"{DB_NAME}.customers_scd2"):
    seed_df = (
        spark.table("customers_silver")
        .withColumn("effective_start_date", F.current_date())
        .withColumn("effective_end_date", F.lit(None).cast("date"))
        .withColumn("is_current", F.lit(True))
    )
    seed_df.write.format("delta").saveAsTable("customers_scd2")

scd2_table = DeltaTable.forName(spark, "customers_scd2")
```

>

[See performance \(1\)](#)

✓ Checked for improvements

```
08:40 AM (4s) 20

current_rows = spark.table("customers_scd2").filter("is_current = true")

changed_df = (
    incremental_df.alias("s")
    .join(current_rows.alias("t"), "customer_id", "left")
    .where(
        F.col("t.customer_id").isNull() | # brand-new customer
        (F.col("s.email") != F.col("t.email")) |
        (F.col("s.city") != F.col("t.city")) |
        (F.col("s.phone") != F.col("t.phone"))
    )
    .select("s.*")
)

print(f"Rows that need a new SCD2 version (changed or new): {changed_df.count()}")
display(changed_df)

> See performance \(2\) ✓ Checked for improvements
> changed_df: pyspark.sql.connect.dataframe.DataFrame = [customer_id: integer, first_name: string ... 5 more fields]
> current_rows: pyspark.sql.connect.dataframe.DataFrame = [customer_id: integer, first_name: string ... 8 more fields]
Rows that need a new SCD2 version (changed or new): 0
```

```
08:40 AM (4s) 21

(
    scd2_table.alias("target")
    .merge(
        changed_df.alias("source"),
        "target.customer_id = source.customer_id AND target.is_current = true"
    )
    .whenMatchedUpdate(set={
        "is_current": "false",
        "effective_end_date": "current_date()"
    })
    .execute()
)

> See performance \(1\) ✓ Checked for improvements
DataFrame[num_affected_rows: bigint, num_updated_rows: bigint, num_deleted_rows: bigint, num_inserted_rows: bigint]
```

```
08:40 AM (6s) 22

new_versions_df = (
    changed_df
    .withColumn("effective_start_date", F.current_date())
    .withColumn("effective_end_date", F.lit(None).cast("date"))
    .withColumn("is_current", F.lit(True))
)

(
    new_versions_df.write
    .format("delta")
    .mode("append")
    .saveAsTable("customers_scd2")
)

print("customers_scd2 total row count (all history):", spark.table("customers_scd2").count())
print("customers_scd2 current row count:", spark.table("customers_scd2").filter("is_current = true").count())
display(spark.table("customers_scd2").orderBy("customer_id", "effective_start_date"))

> See performance \(4\) ✓ Checked for improvements
> new_versions_df: pyspark.sql.connect.dataframe.DataFrame = [customer_id: integer, first_name: string ... 8 more fields]
customers_scd2 total row count (all history): 23
customers_scd2 current row count: 18
```

> [See performance \(4\)](#) ✓ Checked for improvements

> `new_versions_df: pyspark.sql.connect.dataframe.DataFrame = [customer_id: integer, first_name: string ... 8 more fields]`

`customers_scd2 total row count (all history): 23`

`customers_scd2 current row count: 18`

	customer_id	first_name	last_name	email	city	phone	updated_at	effective_start_d
1	1001	John	Doe	john.doe@email.com	New York	555-0101	2024-01-15	2026-07-11
2	1002	Jane	Smith	jane.smith@email.com	Los Angeles	555-0102	2024-01-16	2026-07-11
3	1002	Jane	Smith	jane.smith@newmail.com	San Francisco	555-0102	2024-02-10	2026-07-11
4	1003	Robert	Brown	unknown@example.com	Chicago	555-0103	2024-01-17	2026-07-11
5	1004	Emily	Davis	emily.davis@email.com	Houston	555-0104	2024-01-18	2026-07-11
6	1004	Emily	Davis	emily.davis@newmail.com	Houston	555-0119	2024-02-16	2026-07-11
7	1005	Michael	Wilson	michael.wilson@email.com	Denver	555-0105	2024-02-11	2026-07-11
8	1005	Michael	Wilson	michael.wilson@email.com	Phoenix	555-0105	2024-01-19	2026-07-11
9	1006	Sarah	Taylor	sarah.taylor@email.com	Unknown	555-0106	2024-01-20	2026-07-11
10	1007	David	Anderson	david.anderson@email.com	Philadelphia	555-0107	2024-01-21	2026-07-11
11	1008	Laura	Thomas	unknown@example.com	San Antonio	555-0108	2024-01-22	2026-07-11
12	1009	James	Jackson	james.jackson@email.com	San Diego	555-0109	2024-01-23	2026-07-11
13	1009	James	Jackson	james.jackson@email.com	Seattle	555-0109	2024-02-12	2026-07-11

```

08:40 AM (2s) 23

silver_count = spark.table("customers_silver").count()
scd1_count = spark.table("customers_scd1").count()
scd2_current_count = spark.table("customers_scd2").filter("is_current = true").count()

expected_final_count = len(existing_ids | incoming_ids) # union of old + new customer ids

print(f"Expected distinct customers after merge : {expected_final_count}")
print(f"customers_scd1 row count : {scd1_count}")
print(f"customers_scd2 current-row count : {scd2_current_count}")

assert scd1_count == expected_final_count, "SCD1 row count mismatch!"
assert scd2_current_count == expected_final_count, "SCD2 current-row count mismatch!"
print("Row count validation PASSED")

```

> [See performance \(3\)](#) ✓ Checked for improvements

```

Expected distinct customers after merge : 18
customers_scd1 row count : 18
customers_scd2 current-row count : 18
Row count validation PASSED

```

```

08:40 AM (1s) 24

scd1_dupes = (
    spark.table("customers_scd1")
    .groupBy("customer_id").count()
    .filter("count > 1")
)

scd2_current_dupes = (
    spark.table("customers_scd2")
    .filter("is_current = true")
    .groupBy("customer_id").count()
    .filter("count > 1")
)

```

> `scd1_dupes: pyspark.sql.connect.dataframe.DataFrame = [customer_id: integer, count: long]`

> `scd2_current_dupes: pyspark.sql.connect.dataframe.DataFrame = [customer_id: integer, count: long]`

```

08:40 AM (3s) 25

print(f"Duplicate customer_ids in SCD1: {scd1_dupes.count()}")
print(f"Duplicate customer_ids among SCD2 current rows: {scd2_current_dupes.count()}")

assert scd1_dupes.count() == 0, "Duplicates found in SCD1 table!"
assert scd2_current_dupes.count() == 0, "Duplicates found among SCD2 current rows!"
print("Duplicate validation PASSED")

```

> [See performance \(4\)](#)

08:40 AM (3s)26

```
print("Sample updated customer (id 1002) in SCD1:")
display(spark.table("customers_scd1").filter("customer_id = 1002"))

print("Full history for customer 1002 in SCD2 (should show 2 versions):")
display(spark.table("customers_scd2").filter("customer_id = 1002").orderBy("effective_start_date"))
```

See performance (2)✓ Checked for improvements

Sample updated customer (id 1002) in SCD1:

Table+

🔍🔼📄🗑

	customer_id	first_name	last_name	email	city	phone	updated_at
1	1002	Jane	Smith	jane.smith@newmail.co...	San Francisco	555-0102	2024-02-10

↓

1 row | 2.651s runtime

Refreshed 26 minutes ago

Full history for customer 1002 in SCD2 (should show 2 versions):

08:40 AM (1s)27

```
print("===== FINAL SCD1 TABLE (latest state only) =====")
display(spark.table("customers_scd1").orderBy("customer_id"))
```

See performance (1)✓ Checked for improvements

===== FINAL SCD1 TABLE (latest state only) =====

Table+

🔍🔼📄🗑

	customer_id	first_name	last_name	email	city	phone	updated_at
1	1001	John	Doe	john.doe@email.com	New York	555-0101	2024-01-15
2	1002	Jane	Smith	jane.smith@newmail.com	San Francisco	555-0102	2024-02-10
3	1003	Robert	Brown	unknown@example.com	Chicago	555-0103	2024-01-17
4	1004	Emily	Davis	emily.davis@newmail.com	Houston	555-0119	2024-02-16
5	1005	Michael	Wilson	michael.wilson@email.com	Denver	555-0105	2024-02-11
6	1006	Sarah	Taylor	sarah.taylor@email.com	Unknown	555-0106	2024-01-20
7	1007	David	Anderson	david.anderson@email.com	Philadelphia	555-0107	2024-01-21
8	1008	Laura	Thomas	unknown@example.com	San Antonio	555-0108	2024-01-22

08:40 AM (2s)28

```
print("===== FINAL SCD2 TABLE (full history) =====")
display(spark.table("customers_scd2").orderBy("customer_id", "effective_start_date"))
```

See performance (1)✓ Checked for improvements

===== FINAL SCD2 TABLE (full history) =====

Table+

🔍🔼📄🗑

	customer_id	first_name	last_name	email	city	phone	updated_at
1	1001	John	Doe	john.doe@email.com	New York	555-0101	2024-01-15
2	1002	Jane	Smith	jane.smith@email.com	Los Angeles	555-0102	2024-01-16
3	1002	Jane	Smith	jane.smith@newmail.com	San Francisco	555-0102	2024-02-10
4	1003	Robert	Brown	unknown@example.com	Chicago	555-0103	2024-01-17
5	1004	Emily	Davis	emily.davis@email.com	Houston	555-0104	2024-01-18
6	1004	Emily	Davis	emily.davis@newmail.com	Houston	555-0119	2024-02-16
7	1005	Michael	Wilson	michael.wilson@email.com	Denver	555-0105	2024-02-11